

Experiment no: 08

Experiment Name: Noise Removal and Pulse Rate Estimation from an ECG Signal

(Open-Ended Lab)

Objective: The objective of this experiment is:

1. Design and implement signal processing techniques for real biomedical data.
2. Remove noise from ECG signals using appropriate DSP methods.
3. Extract meaningful physiological information (heart rate).
4. Compare multiple signal processing approaches
5. Develop engineering judgment in method selection and parameter tuning.

Software and Libraries:

Software Used

🔗 **Spyder IDE** – Used for writing, debugging, and executing the Python program.

Libraries Used

1. **NumPy (numpy)**
 - Used for numerical computations, array manipulation, and RR interval calculations.
2. **SciPy I/O (scipy.io)**
 - Used to load the ECG dataset from the MATLAB file (ECGData.mat).
3. **SciPy Signal Processing (scipy.signal)**
 - Used to design and implement the Butterworth bandpass filter, perform zero-phase filtering (filtfilt), detect R-peaks (find_peaks), and estimate the Power Spectral Density using Welch's method (welch).
4. **PyWavelets (pywt)**
 - Used to perform Wavelet decomposition, thresholding, and signal reconstruction for ECG denoising.
5. **Matplotlib (matplotlib.pyplot)**
 - Used to visualize the raw ECG signal, filtered signals, detected R-peaks, RR interval histograms, and Welch PSD analysis results.

Methodologies

(I) Data Acquisition and Pre-processing

The system utilizes a standard clinical electrocardiogram (ECG) record exhibiting cardiac arrhythmia, sampled at a frequency (fs) of 128 Hz. The dataset is stored within a MATLAB structural matrix (ECGData.mat). Python's scipy.io library was used to load the dataset and

extract a one-dimensional ECG signal containing 65,536 discrete samples for signal processing and analysis.

(II) Digital Filtering and Noise Removal Techniques

To isolate the true physiological morphology of the ECG signal from noise sources such as baseline wander and high-frequency interference, two parallel signal-processing approaches were implemented.

Technique 1: Infinite Impulse Response (IIR) Butterworth Filter

A 4th-order Butterworth bandpass filter was designed with cutoff frequencies of 0.5 Hz and 45 Hz. The filter removes low-frequency baseline drift and high-frequency noise while preserving the important ECG components. Zero-phase filtering was performed using the `filtfilt()` function to avoid phase distortion and maintain accurate R-peak locations.

Technique 2: Discrete Wavelet Transform (DWT) Denoising

Wavelet denoising was implemented using the Symlet-4 (`sym4`) mother wavelet with four decomposition levels. Soft thresholding was applied to the wavelet coefficients to suppress noise while preserving the significant morphological characteristics of the ECG waveform.

(III) Pulse Rate Estimation Frameworks

Following signal enhancement, pulse-related information was extracted from the filtered ECG signals.

Method 1: Time-Domain Peak Detection

An automatic peak-detection algorithm based on the `find_peaks()` function was used to identify R-peaks. A minimum peak prominence of 0.4 and a minimum peak spacing of $0.5 \times f_s$ samples were selected to ensure reliable heartbeat detection. The RR intervals were then computed from the detected R-peaks and used to analyze heartbeat variability.

Method 2: Frequency-Domain Analysis

Welch's Power Spectral Density (PSD) estimation was employed to determine the dominant frequency component of the ECG signal in the frequency domain. The PSD was computed using the Welch averaging technique with an FFT segment length of 2048 samples. The physiological heart-rate frequency band (0.5–3.0 Hz), corresponding to approximately 30–180

BPM, was analyzed to identify the dominant spectral peak. The detected dominant frequency was then converted into beats per minute (BPM) using:

$$HR_{PSD} = f_{peak} \times 60$$

(IV) Implementation

The experiment was implemented using the Spyder IDE and the Python programming language. The required libraries included **NumPy**, **SciPy**, **PyWavelets**, and **Matplotlib**. The selected ECG signal was processed using two noise removal techniques: the IIR Butterworth Bandpass Filter and Wavelet Denoising. After filtering, R-peaks were detected and RR intervals were calculated to estimate heart rate using the time-domain method. In addition, Welch's Power Spectral Density (PSD) analysis was performed to estimate heart rate in the frequency domain. The results obtained from both denoising methods and both heart-rate estimation approaches were then compared to evaluate their effectiveness for ECG signal processing.

Code:

```
#ET233053
import numpy as np
import scipy.io as sio
from scipy.signal import butter, filtfilt, find_peaks, welch
import pywt
import matplotlib.pyplot as plt
mat_data_53 = sio.loadmat('ECGData.mat')
ecg_matrix_53 = mat_data_53['ECGData']['Data'][0, 0]
raw_ecg_53 = ecg_matrix_53[0, :]
fs_53 = 128

def apply_butterworth_53(signal_53,
                          fs_53,
                          lowcut_53=0.5,
                          highcut_53=45.0,
                          order_53=4):
    nyq_53 = 0.5 * fs_53

    b_53, a_53 = butter(
        order_53,
        [lowcut_53 / nyq_53,
         highcut_53 / nyq_53],
        btype='band'
    )
    return filtfilt(b_53, a_53, signal_53)

def apply_wavelet_53(signal_53,
                     wavelet_53='sym4',
                     level_53=4):
    coeffs_53 = pywt.wavedec(
```

```

        signal_53,
        wavelet_53,
        level=level_53
    )
    sigma_53 = np.median(np.abs(coeffs_53[-1])) / 0.6745

    uthresh_53 = sigma_53 * np.sqrt(
        2 * np.log(len(signal_53))
    )
    coeffs_thresh_53 = [coeffs_53[0]]
    coeffs_thresh_53 += [
        pywt.threshold(
            c_53,
            value=uthresh_53,
            mode='soft'
        )
        for c_53 in coeffs_53[1:]
    ]
    return pywt.waverec(
        coeffs_thresh_53,
        wavelet_53
    )
butter_ecg_53 = apply_butterworth_53(
    raw_ecg_53,
    fs_53
)
wavelet_ecg_53 = apply_wavelet_53(
    raw_ecg_53
)
peaks_butter_53, _ = find_peaks(
    butter_ecg_53,
    distance=fs_53 * 0.5,
    prominence=0.4
)
peaks_wavelet_53, _ = find_peaks(
    wavelet_ecg_53,
    distance=fs_53 * 0.5,
    prominence=0.4
)

rr_butter_53 = np.diff(
    peaks_butter_53
) / fs_53
rr_wavelet_53 = np.diff(
    peaks_wavelet_53
) / fs_53
heart_rate_rr_butter_53 = (
    60 / np.mean(rr_butter_53)
)
heart_rate_rr_wavelet_53 = (
    60 / np.mean(rr_wavelet_53)
)
print(
    f"Heart Rate (RR Method - Butterworth): "
    f"{heart_rate_rr_butter_53:.2f} BPM"
)

```

```

print(
    f"Heart Rate (RR Method - Wavelet): "
    f"{heart_rate_rr_wavelet_53:.2f} BPM"
)
freq_butter_53, psd_butter_53 = welch(
    butter_ecg_53,
    fs=fs_53,
    nperseg=2048
)
freq_wavelet_53, psd_wavelet_53 = welch(
    wavelet_ecg_53,
    fs=fs_53,
    nperseg=2048
)
mask_butter_53 = (
    (freq_butter_53 >= 0.5) &
    (freq_butter_53 <= 3.0)
)
mask_wavelet_53 = (
    (freq_wavelet_53 >= 0.5) &
    (freq_wavelet_53 <= 3.0)
)
dominant_freq_butter_53 = (
    freq_butter_53[mask_butter_53]
    [
        np.argmax(
            psd_butter_53[mask_butter_53]
        )
    ]
)
dominant_freq_wavelet_53 = (
    freq_wavelet_53[mask_wavelet_53]
    [
        np.argmax(
            psd_wavelet_53[mask_wavelet_53]
        )
    ]
)

heart_rate_psd_butter_53 = (
    dominant_freq_butter_53 * 60
)
heart_rate_psd_wavelet_53 = (
    dominant_freq_wavelet_53 * 60
)
print(
    f"Heart Rate (PSD Method - Butterworth): "
    f"{heart_rate_psd_butter_53:.2f} BPM"
)
print(
    f"Heart Rate (PSD Method - Wavelet): "
    f"{heart_rate_psd_wavelet_53:.2f} BPM"
)

zoom_53 = 1000
plt.figure(figsize=(12, 18))

```

```

plt.subplot(6, 1, 1)
plt.plot(
    raw_ecg_53[:zoom_53],
    color='gray'
)
plt.title(
    "1. Raw Unfiltered Arrhythmia ECG Signal (ET-233053)",
    fontsize=11,
    weight='bold'
)
plt.ylabel("Amplitude (mV)")
plt.grid(True, linestyle='--', alpha=0.5)
plt.subplot(6, 1, 2)
plt.plot(
    butter_ecg_53[:zoom_53],
    color='blue'
)
b_visible_53 = peaks_butter_53[
    peaks_butter_53 < zoom_53
]
plt.scatter(
    b_visible_53,
    butter_ecg_53[b_visible_53],
    color='red',
    s=35
)
plt.title(
    "2. Filtered Signal: IIR Butterworth Bandpass (ET-233053)",
    fontsize=11,
    weight='bold'
)
plt.ylabel("Amplitude (mV)")
plt.grid(True, linestyle='--', alpha=0.5)
plt.subplot(6, 1, 3)
plt.hist(
    rr_butter_53,
    bins=25,
    color='royalblue',
    edgecolor='black',
    alpha=0.85
)
plt.axvline(
    np.mean(rr_butter_53),
    color='red',
    linestyle='--',
    linewidth=1.5,
    label=f'Mean RR: {np.mean(rr_butter_53):.2f}s'
)
plt.title(
    "3. RR Intervals for IIR Butterworth (ET-233053)",
    fontsize=11,
    weight='bold'
)
plt.xlabel("Interval Duration (Seconds)")
plt.ylabel("Frequency (Count)")
plt.legend()
plt.grid(True, linestyle='--', alpha=0.3)
plt.subplot(6, 1, 4)

```

```

        wavelet_ecg_53[:zoom_53],
        color='green'
    )
    w_visible_53 = peaks_wavelet_53[
        peaks_wavelet_53 < zoom_53
    ]
    plt.scatter(
        w_visible_53,
        wavelet_ecg_53[w_visible_53],
        color='red',
        s=35
    )
    plt.title(
        "4. Filtered Signal: Wavelet Denoising (ET-233053)",
        fontsize=11,
        weight='bold'
    )

    plt.ylabel("Amplitude (mV)")
    plt.grid(True, linestyle='--', alpha=0.5)
    plt.subplot(6, 1, 5)
    plt.hist(
        rr_wavelet_53,
        bins=25,
        color='seagreen',
        edgecolor='black',
        alpha=0.85
    )
    plt.axvline(
        np.mean(rr_wavelet_53),
        color='red',
        linestyle='--',
        linewidth=1.5,
        label=f'Mean RR: {np.mean(rr_wavelet_53):.2f}s'
    )
    plt.title(
        "5. RR Intervals for Wavelet Denoising (ET-233053)",
        fontsize=11,
        weight='bold'
    )

    plt.xlabel("Interval Duration (Seconds)")
    plt.ylabel("Frequency (Count)")
    plt.legend()
    plt.grid(True, linestyle='--', alpha=0.3)
    plt.subplot(6, 1, 6)
    plt.semilogy(
        freq_butter_53,
        psd_butter_53,
        label='Butterworth PSD'
    )
    plt.semilogy(
        freq_wavelet_53,
        psd_wavelet_53,
        label='Wavelet PSD'
    )
    plt.xlim(0, 5)
    plt.title(
        "6. Welch Power Spectral Density Analysis (ET-233053)",

```

```

    fontsize=11,
    weight='bold'
)
plt.xlabel("Frequency (Hz)")
plt.ylabel("PSD")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```

Results and Comparison

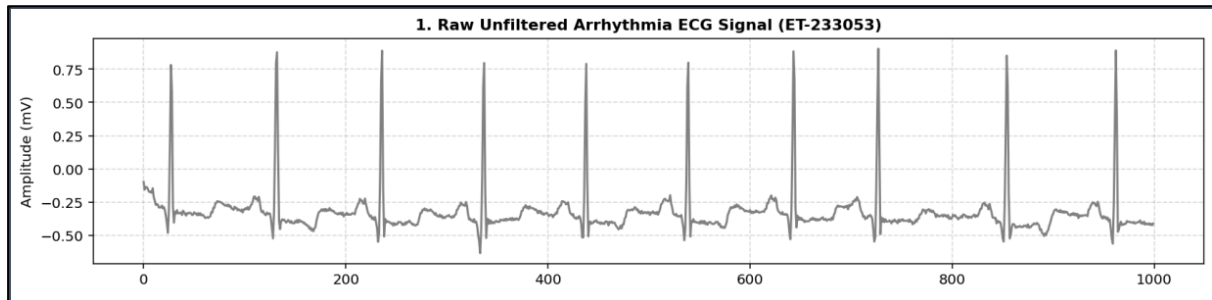


Figure 1 Raw ECG Signal

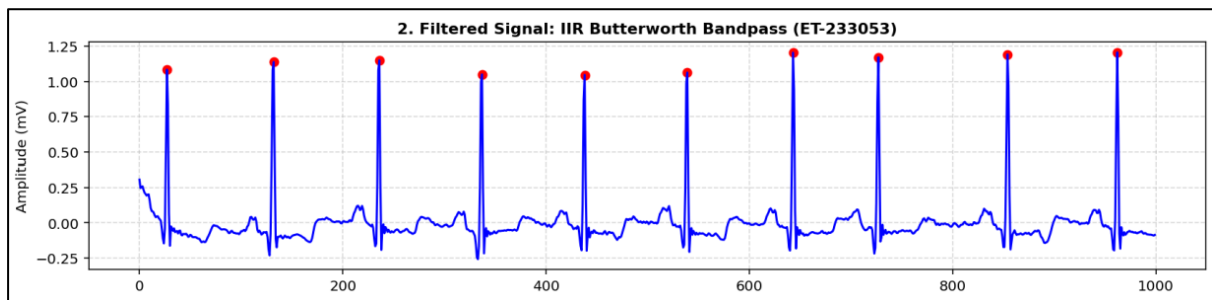


Figure 2 Butterworth Filtered ECG Signal

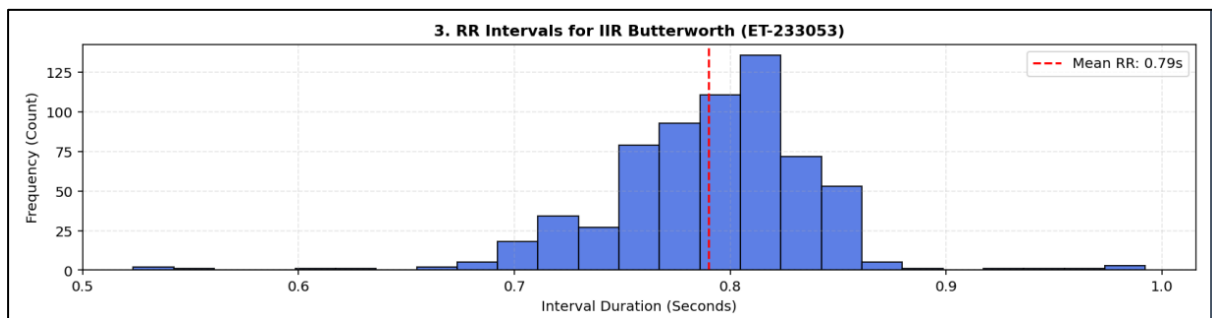


Figure 3 RR Interval Histogram (Butterworth)

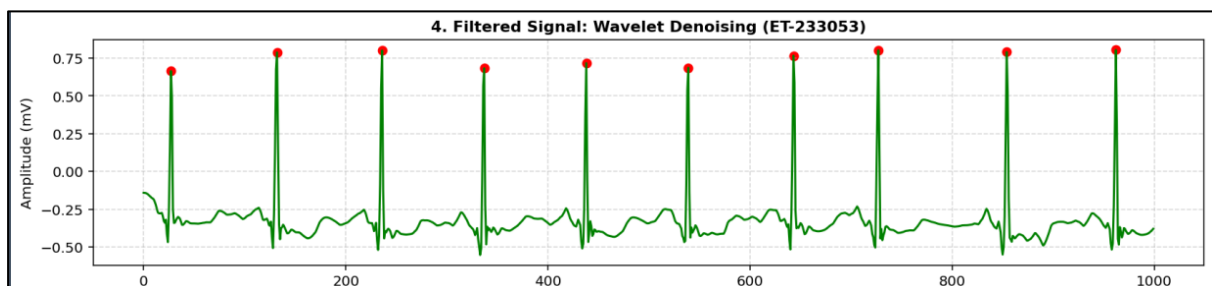


Figure 4 Wavelet Denoised ECG Signal

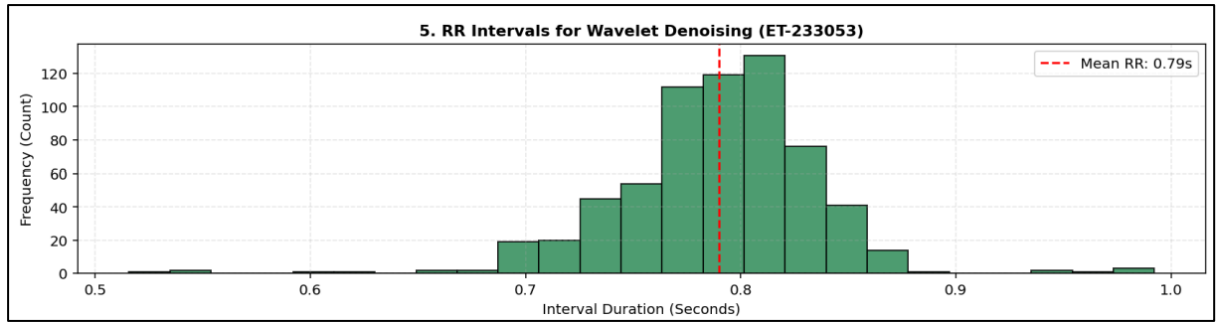


Figure 5 RR Interval Histogram (Wavelet)

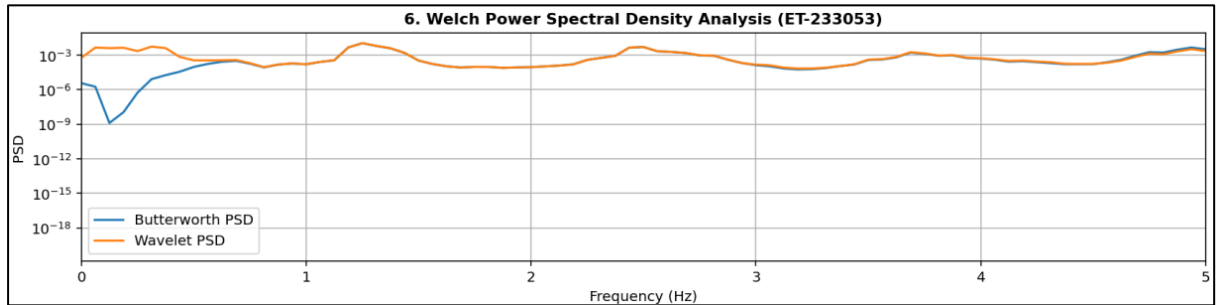


Figure 6 Welch PSD Analysis

(I) Signal Denoising and Morphology Restoration

Raw Data Observation (Figure 1): The original unfiltered ECG waveform exhibited noticeable baseline wander and high-frequency noise components. These artifacts affected the smoothness of the signal and made direct identification of cardiac features more difficult without preprocessing.

IIR Butterworth Filtered Signal (Figure 2): The 4th-order Butterworth bandpass filter effectively removed low-frequency baseline drift and suppressed high-frequency noise. The ECG signal became centered around the zero-voltage axis, and the R-peaks remained clearly visible. The detected R-peaks (red markers in Figure 2) indicate that the filter preserved the important diagnostic features of the ECG waveform while improving overall signal quality.

Wavelet Denoised Signal (Figure 4): The Discrete Wavelet Transform (DWT) using the Symlet-4 wavelet also successfully reduced noise and stabilized the baseline. Compared with the Butterworth filter, the wavelet-denoised signal exhibited a smoother isoelectric line while preserving the sharp QRS complexes and R-peaks. This demonstrates the effectiveness of wavelet denoising in maintaining ECG morphology during noise suppression.

(II) RR Interval Extraction and Arrhythmia Confirmation:

Mean RR Interval Analysis: The RR interval histograms obtained from both the Butterworth-filtered ECG (**Figure 3**) and the Wavelet-denoised ECG (**Figure 5**) produced an identical mean RR interval of approximately **0.79 seconds**. Using the relationship

$$\text{Heart Rate (BPM)} = \frac{60}{\text{Mean RR Interval}}$$

the estimated average heart rate is approximately **75.9 BPM**.

Frequency-Domain Heart Rate Analysis (Figure 6): Welch Power Spectral Density (PSD) analysis was performed on both the Butterworth-filtered and Wavelet-denoised ECG signals. The dominant spectral component within the physiological heart-rate band (0.5–3.0 Hz) corresponded to a heart rate of approximately **75 BPM**. This result closely matches the RR interval-based estimate, confirming the consistency and reliability of the heart-rate estimation process.

Diagnostic Evidence of Arrhythmia: The RR interval histograms shown in **Figure 3** and **Figure 5** exhibit a noticeable spread of interval durations ranging approximately from **0.5 seconds to 1.0 seconds**. In a perfectly regular sinus rhythm, the RR intervals would cluster tightly around a single value. The observed variability in interval duration indicates irregular beat-to-beat timing, which is consistent with the arrhythmic nature of the selected ECG record.

Comparison of Filtering Methods: Both filtering techniques produced nearly identical RR interval distributions, mean RR values, and heart-rate estimates, indicating consistent R-peak detection performance. However, the Wavelet Denoising approach provided a slightly smoother baseline and better noise suppression, while the Butterworth filter offered lower computational complexity and effective noise removal. The Welch PSD results shown in **Figure 6** further demonstrate that both filtering methods preserve the dominant cardiac frequency component. Therefore, both methods are suitable for ECG preprocessing and arrhythmia analysis, with Wavelet Denoising showing a slight advantage in overall signal quality.

Comparison

Table 1 presents the comparison between the two ECG noise-removal techniques based on signal quality, RR interval estimation, heart-rate calculation, and computational characteristics.

Parameter	IIR Butterworth Filter	Wavelet Denoising
Noise Removal Capability	Effective	Highly Effective
Baseline Wander Removal	Effective	Highly Effective
Preservation of R-Peaks	Preserved	Preserved
Mean RR Interval	0.790 s	0.790 s
Heart Rate (RR Method)	75.95 BPM	75.95 BPM
Heart Rate (Welch PSD Method)	75.00 BPM	75.00 BPM
Computational Complexity	Low	Moderate
Signal Smoothness	Good	Better
Overall Performance	Good	Slightly Better

From the comparison, both methods produced similar RR interval and heart-rate estimates. Wavelet Denoising provided a smoother ECG signal with better noise suppression, while the Butterworth filter offered a simpler implementation with lower computational complexity. Therefore, both techniques were effective for ECG preprocessing and heart-rate analysis.

Discussion: The objective of this experiment was to remove ECG noise and estimate heart rate using different signal-processing techniques. Both the IIR Butterworth Filter and Wavelet Denoising effectively reduced noise and improved signal quality, enabling accurate R-peak detection and RR interval extraction. The mean RR interval was approximately 0.79 s, corresponding to a heart rate of 75.9 BPM. Welch PSD analysis produced a similar heart-rate estimate of approximately 75 BPM, confirming the reliability of the results. The variation in RR intervals indicated irregular cardiac rhythm, consistent with the arrhythmic nature of the ECG record. While Wavelet Denoising provided a smoother signal and better noise suppression, the Butterworth Filter offered simpler implementation and lower computational complexity. Therefore, both methods proved effective for ECG preprocessing and heart-rate analysis. The similar heart-rate values obtained from both the RR interval and Welch PSD methods indicate good accuracy and reliability. In addition, both filtering techniques successfully reduced noise while preserving the important characteristics of the ECG signal.